

MOBILE DEVELOPMENT – LAB WORK

Moving between screens

Navigation & routing in Flutter

Navigator `push`, `pushNamed`, `pop`, simple page transitions and `go_router`.

Format	Hands-on, individual work
Tasks	6 categories × 5 tasks
Samples	Task X.1 of each category is solved
Tools	Flutter SDK, emulator or device

WHAT YOU WILL LEARN

By the end of this lab you will be able to:

- move between screens with `Navigator.push` and go back with `pop`;
- define and use *named routes* (`pushNamed`, `onGenerateRoute`);
- pass data forward (arguments) and backward (results) between screens;
- write simple custom page transitions (fade, slide, scale);
- build declarative navigation with the `go_router` package.

2 HOURS JUNIORS

How this lab is organised

The lab has **6 categories**. Every category contains **5 tasks**. Task **X.1** of each category is a **sample with a full solution**: read it, type it in (do not copy-paste!), run it and make sure you understand every line. Tasks **X.2–X.5** are for you to solve. Difficulty is shown with stars: ★ easy, ★★ medium, ★★★ challenging.

Suggested time plan (120 minutes)

Part	Content	Time
Setup	Create the project, run on emulator/device	10 min
Category 1	Navigator: `push` and `pop`	20 min
Category 2	Named routes	15 min
Category 3	Passing arguments with constructors, returning results	20 min
Category 4	Passing arguments with named routes	15 min
Category 5	Simple transition animations	15 min
Category 6	`go_router`	20 min
Wrap-up	Self-check and questions	5 min

> **Realistic goal.** In two hours you are not expected to finish all 30 tasks. **Minimum:** the six samples plus tasks X.2 and X.3 from every category. Tasks X.4 and X.5 are *bonus / homework*.

Setup (10 min)

1. Create a project and open it in your IDE:

```
flutter create nav_lab
cd nav_lab
flutter pub add go_router      # needed in Category 6
flutter run
```

2. Keep one file `lib/main.dart` per task (or one folder per category). Replace the whole file for each new sample.
3. Use **hot reload** (`r`) after small changes and **hot restart** (`R`) after changing `main()` or the route table.

KEY IDEA: THE NAVIGATION STACK

Flutter keeps screens (routes) in a *stack*. `push` puts a new screen on top, `pop` removes the top screen and reveals the one below. The `AppBar` shows a back arrow automatically when there is something to go back to.

01 / Navigator: push and pop

QUICK REFERENCE

`Navigator.push(context, route)`	Open a new screen on top of the stack.
`Navigator.pop(context)`	Close the current screen.
`Navigator.pushReplacement(...)`	Replace the current screen with a new one.
`Navigator.popUntil(context, test)`	Pop screens until the test returns true.
`Navigator.canPop(context)`	Is there a screen to go back to?

Task 1.1 Two screens with push and pop

SOLVED EXAMPLE

Goal. Create a `HomeScreen` with a button that opens a `DetailsScreen`. The second screen has a button that returns back.

Solution

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: HomeScreen()));

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Open details'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const DetailsScreen()),
            );
          },
        ),
      ),
    );
  }
}

class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Details')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Go back'),
          onPressed: () => Navigator.pop(context),
        ),
      ),
    );
  }
}
```

Observe: the `AppBar` of `DetailsScreen` shows a back arrow without any extra code. The system back button does the same as pop.

Task 1.2 Three-screen chain ★ 10 pts

Create screens **A**, **B** and **C**. Each screen has a different `AppBar` colour and a button *Next* that opens the next screen (A → B → C). Screen C has a button *Back* that returns to B.

- Check what happens when you press the system back button on each screen.

Task 1.3 Replace, do not stack ★ 10 pts

Build a *Login* screen with a button *Sign in*. After pressing it the user lands on *Home* and **cannot** go back to *Login*.

Hint. Use `Navigator.pushReplacement`. Compare the behaviour with a normal `push`.

Task 1.4 Back to the start ★★ 15 pts

Make a flow of four screens (Step 1 → Step 4). On Step 4 add a button *Finish* that returns directly to Step 1, removing Steps 2 to 4 from the stack at once.

Hint. `Navigator.popUntil(context, (route) => route.isFirst);`

Task 1.5 Smart back button ★★ 15 pts

Create a reusable widget `BackOrExitButton`. It shows the text *Back* if `Navigator.canPop(context)` is true, otherwise it is disabled and shows *Nothing to go back to*. Use it on a root screen and on a pushed screen and verify both cases.

02 / Named routes

QUICK REFERENCE

`routes: {'/': (c) => ...}`	Route table inside `MaterialApp`.
`initialRoute: '/'`	Screen shown at app start.
`Navigator.pushNamed(context, '/x')`	Open a screen by name.
`Navigator.pushReplacementNamed(...)`	Replace the current screen by name.
`onUnknownRoute`	Fallback when a name is not found.

Task 2.1 Route table and pushNamed SOLVED EXAMPLE

Goal. Replace `MaterialPageRoute` with names. Keep all route names in one place to avoid typos.

Solution

```dart
import 'package:flutter/material.dart';

```

```dart
class AppRoutes {
  static const home = '/';
  static const settings = '/settings';
}

void main() {
  runApp(MaterialApp(
    initialRoute: AppRoutes.home,
    routes: {
      AppRoutes.home: (context) => const HomeScreen(),
      AppRoutes.settings: (context) => const SettingsScreen(),
    },
  ));
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Settings'),
          onPressed: () => Navigator.pushNamed(context, AppRoutes.settings),
        ),
      ),
    );
  }
}

class SettingsScreen extends StatelessWidget {
  const SettingsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Settings')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Back'),
          onPressed: () => Navigator.pop(context),
        ),
      ),
    );
  }
}
```

Observe: the screens do not need to know about each other's classes any more, only about the route *name*.

Task 2.2 Menu with four routes ★ 10 pts

Create routes `/`, `/profile`, `/messages`, `/about`. The home screen contains a `ListView` with three `ListTiles` that open the other screens with `pushNamed`.

Task 2.3 Page not found ★★ 15 pts

Add a `NotFoundScreen` and register it with `onUnknownRoute`. Add a button on Home that

calls `pushNamed(context, '/does-not-exist')` and check that your 404 screen opens.

Hint. `onUnknownRoute: (settings) => MaterialPageRoute(builder: (c) => const NotFoundScreen())`

Task 2.4 Drawer navigation ★★ 15 pts

Add a `Drawer` to the home screen with entries for all routes. Use `pushReplacementNamed` for the drawer items so the stack does not grow when the user switches sections.

- Close the drawer before navigating (`Navigator.pop(context)`).

Task 2.5 Log out ★★★ 20 pts

Create `/login`, `/home`, `/profile`. From *Profile* the button *Log out* must go to `/login` and **clear the whole stack**.

Hint. `Navigator.pushNamedAndRemoveUntil(context, '/login', (route) => false);`

03 / Passing arguments with constructors

QUICK REFERENCE

The simplest and *type-safe* way to send data forward is a constructor parameter of the next screen. To send data *back*, call `Navigator.pop(context, value)` and wait for it with `final result = await Navigator.push(...);`.

Task 3.1 Product list to product details SOLVED EXAMPLE

Goal. Show a list of products. When the user taps one, open a details screen that displays the name and the price of the selected product.

Solution

```dart
import 'package:flutter/material.dart';

class Product {
  final String name;
  final double price;
  const Product(this.name, this.price);
}

const products = [
  Product('Laptop', 999.0),
  Product('Phone', 599.5),
  Product('Headphones', 79.9),
];

void main() => runApp(const MaterialApp(home: ProductListScreen()));

class ProductListScreen extends StatelessWidget {
  const ProductListScreen({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: const Text('Products')),
      body: ListView(
        children: [
          for (final p in products)
            ListTile(
              title: Text(p.name),
              subtitle: Text('$${p.price}'),
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => ProductScreen(product: p),
                  ),
                );
              },
            ),
        ],
      ),
    );
  }
}

class ProductScreen extends StatelessWidget {
  final Product product;
  const ProductScreen({super.key, required this.product});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(product.name)),
      body: Center(
        child: Text('Price: $${product.price}',
            style: const TextStyle(fontSize: 28)),
      ),
    );
  }
}
```

Observe: if you forget `required` data, the compiler tells you – this is the biggest advantage of constructor arguments.

Task 3.2 Greeting ★ 10 pts

Screen 1 has a `TextField` for a name and a button *Greet*. Screen 2 shows *Hello, <name>!* using the text passed through its constructor.

Task 3.3 Contact card ★★ 15 pts

Create a `Contact` class (`name`, `phone`, `email`) and a list of 5 contacts. Tapping a contact opens a `ContactScreen` that shows all three fields in a styled card. Pass the whole object, not three separate strings.

Task 3.4 Pick a colour (result back) ★★ 20 pts

Screen 1 shows a square with the current colour and a button *Choose colour*. Screen 2 shows three coloured buttons. When one is tapped, screen 2 closes and screen 1 updates the

square.

Hint. On screen 2: `Navigator.pop(context, Colors.red);` On screen 1: `final c = await Navigator.push<Color>(...);` and then `setState`. Use a `StatefulWidget`.

Task 3.5 Edit a counter ★★★ 20 pts

Screen 1 stores an integer. Pass it to an *Editor* screen that has + and – buttons. When the user presses *Save*, return the new value; when the user presses the back arrow, **keep** the old value (the result will be `null`).

- Handle the `null` result correctly.

04 / Passing arguments with named routes

QUICK REFERENCE

With named routes, data travels in the `arguments` parameter (type `Object?`):
`Navigator.pushNamed(context, '/details', arguments: myObject);`
On the receiving screen:
`final args = ModalRoute.of(context)!.settings.arguments as MyClass;`
Because `arguments` is untyped, always use a small class and cast carefully.

Task 4.1 ScreenArguments class SOLVED EXAMPLE

Goal. Open `/details` and pass a title and a message packed in a class.

Solution

```dart
import 'package:flutter/material.dart';

class ScreenArguments {
  final String title;
  final String message;
  const ScreenArguments(this.title, this.message);
}

void main() {
  runApp(MaterialApp(
    initialRoute: '/',
    routes: {
      '/': (context) => const HomeScreen(),
      '/details': (context) => const DetailsScreen(),
    },
  ));
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: ElevatedButton(
```

```
            child: const Text('Send arguments'),
            onPressed: () {
              Navigator.pushNamed(
                context,
                '/details',
                arguments: const ScreenArguments('Welcome', 'Sent from Home'),
              );
            },
          ),
        ),
      );
  }
}

class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final args =
        ModalRoute.of(context)!.settings.arguments as ScreenArguments;
    return Scaffold(
      appBar: AppBar(title: Text(args.title)),
      body: Center(child: Text(args.message)),
    );
  }
}
```

Observe: the arguments are read inside `build`, using the `context` of the screen.

Task 4.2 Send a number ★ 10 pts

Home has three buttons labelled 1, 2, 3. Each pushes `/square` with the number as an argument (plain `int`). The `/square` screen shows the number and its square.

Task 4.3 User profile ★★ 15 pts

Create a `User` class (`name`, `age`, `city`). Show a list of 4 users. Tapping a user opens `/profile` with the `User` object as an argument. Display all fields.

Task 4.4 onGenerateRoute ★★★ 20 pts

Remove the `routes` table and use `onGenerateRoute` instead. For `/profile`, read the `User` from `settings.arguments`, check its type, and create `ProfileScreen(user: user)` using the **constructor**.

Hint. `onGenerateRoute: (settings) { if (settings.name == '/profile') { final user = settings.arguments as User; return MaterialPageRoute(builder: (c) => ProfileScreen(user: user)); } return null; }`

Task 4.5 Result from a named route ★★★ 20 pts

Open `/rate` with the argument *movie title*. The `/rate` screen has five star buttons. After tapping a star it returns the rating. Home shows *Movie: 4/5*.

- Use `final rating = await Navigator.pushNamed<int>(...)`.

- What does `Home` display if the user returns with the back arrow?

05 / Simple transition animations

QUICK REFERENCE

`PageRouteBuilder` lets you describe *how* a screen appears. It needs two builders: `pageBuilder` (the screen) and `transitionsBuilder` (the animation). The `animation` value goes from 0.0 to 1.0 while the screen is opening and back from 1.0 to 0.0 while it is closing.

`FadeTransition`	Changes opacity.
`SlideTransition`	Moves the screen using an `Offset` tween.
`ScaleTransition`	Grows or shrinks the screen.
`Curves.easeInOut`	Makes motion feel natural.

Task 5.1 Fade transition

SOLVED EXAMPLE

Goal. Open the details screen with a smooth fade-in that lasts 400 ms.

Solution

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: HomeScreen()));

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Open with fade'),
          onPressed: () {
            Navigator.push(
              context,
              PageRouteBuilder(
                transitionDuration: const Duration(milliseconds: 400),
                pageBuilder: (context, animation, secondaryAnimation) =>
                    const DetailsScreen(),
                transitionsBuilder:
                    (context, animation, secondaryAnimation, child) {
                  return FadeTransition(opacity: animation, child: child);
                },
              ),
            );
          },
        ),
      ),
    );
  }
}

```

```dart
class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Details')),
      body: const Center(child: Text('I faded in!')),
    );
  }
}
```

Observe: press back – the same animation plays in reverse automatically.

Task 5.2 Slide from the right ★ 10 pts

Change the sample so the new screen slides in from the right edge.

Hint. `SlideTransition(position: Tween<Offset>(begin: const Offset(1, 0), end: Offset.zero).animate(animation), child: child)`

Task 5.3 Slide from the bottom ★★ 15 pts

Create a second button *Open as a sheet* that makes the screen slide in from the bottom (`Offset(0, 1)`). Use `CurvedAnimation` with `Curves.easeOut`. Set a duration of 500 ms.

Task 5.4 Scale and fade together ★★ 15 pts

Combine `ScaleTransition` and `FadeTransition` (nest one inside the other) so the screen grows from 80% to 100% while becoming visible.

Hint. `Tween<double>(begin: 0.8, end: 1.0).animate(animation)`

Task 5.5 Reusable route and global theme ★★★ 20 pts

1. Create a class `FadeRoute extends PageRouteBuilder` with a constructor `FadeRoute(Widget page)` so any push becomes `Navigator.push(context, FadeRoute(const DetailsScreen()))`.
2. Then, *without* custom routes, change the transition for the whole app using `ThemeData(pageTransitionsTheme: ...)` with `CupertinoPageTransitionsBuilder` on Android. Compare the result.

06 / go_router

QUICK REFERENCE

`go_router` describes navigation *declaratively* with URL-like paths. Add it with `flutter pub add go_router` and import `package:go_router/go_router.dart`.

`context.go('/details')`	Go to a location, *replacing* the current stack.
`context.push('/details')`	Push on top of the current stack (like `Navigator.push`).
`context.pop()`	Go back one screen.
`path: '/item/:id'`	Path parameter, read via `state.pathParameters['id']`.
`context.push('/x', extra: obj)`	Pass an object, read via `state.extra`.
`context.goNamed('item', pathParameters: {...})`	Navigate by route name.

Task 6.1 First router: go, push and pop SOLVED EXAMPLE

Goal. Create a router with two routes. Home opens Details using both `go` and `push`, and you observe the difference.

Solution

```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

final router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const HomeScreen(),
    ),
    GoRoute(
      path: '/details',
      builder: (context, state) => const DetailsScreen(),
    ),
  ],
);

void main() => runApp(MaterialApp.router(routerConfig: router));

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            ElevatedButton(
              onPressed: () => context.go('/details'),
              child: const Text('go (replace stack)'),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: () => context.push('/details'),
              child: const Text('push (add to stack)'),
            ),
          ],
        ),
```

```
      ),
    );
  }
}

class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Details')),
      body: Center(
        child: ElevatedButton(
          onPressed: () => context.pop(),
          child: const Text('pop'),
        ),
      ),
    );
  }
}
```

Observe: after `go`, the `AppBar` may have no back arrow and `context.pop()` has nothing to pop. After `push`, the arrow is visible and `pop` works. Why?

Task 6.2 Path parameters ★ 10 pts

Add a route `/product/:id`. The home screen shows buttons *Product 1*, *2*, *3* that call `context.push('/product/1')` etc. The product screen displays *Product #id* read from `state.pathParameters['id']`.

Task 6.3 Passing an object with extra ★★ 15 pts

Reuse the `Product` class from Category 3. Show a list of products. On tap call `context.push('/product-details', extra: product)`. In the route builder obtain it with `state.extra as Product` and pass it to `ProductScreen(product: ...)`.

- What happens with `extra` when the app is reloaded in a browser? Why is a path parameter safer?

Task 6.4 Named routes and 404 page ★★ 15 pts

Give each route a `name` (`'home'`, `'settings'`, `'item'`) and navigate with `context.goNamed(...)`. For `'item'` send the id with `pathParameters: {'id': '42'}`. Add an `errorBuilder` that shows a *Page not found* screen.

Task 6.5 Transition in go_router ★★★ 20 pts

Replace `builder` with `pageBuilder` for the details route and return a `CustomTransitionPage` with a slide-from-right transition.

```
GoRoute(
  path: '/details',
  pageBuilder: (context, state) => CustomTransitionPage(
    key: state.pageKey,
    child: const DetailsScreen(),
    transitionsBuilder: (context, animation, secondaryAnimation, child) {
```

```
      return FadeTransition(opacity: animation, child: child); // change me
    },
  ),
),
```

- Replace the fade with a slide and a scale transition from Category 5.
- Move the transition into a helper function `buildPage(Widget child)` and reuse it for all routes.

Self-check and progress sheet

Mark the tasks you completed and ask your teacher to sign off.

Category	X.1	X.2	X.3	X.4	X.5
1. Push and pop	☐	☐	☐	☐	☐
2. Named routes	☐	☐	☐	☐	☐
3. Constructor arguments	☐	☐	☐	☐	☐
4. Named-route arguments	☐	☐	☐	☐	☐
5. Transitions	☐	☐	☐	☐	☐
6. go_router	☐	☐	☐	☐	☐

Questions to answer in your own words

1. What is the difference between `push` and `pushReplacement`?
2. Why should route names be kept in constants?
3. How do you return a value from a screen, and what is received if the user presses the system back button?
4. What is the difference between `context.go` and `context.push` in `go_router`?
5. When is a path parameter better than `extra`?

Grading

Minimum set (6 samples + tasks X.2, X.3)	up to 70 pts
Bonus tasks X.4, X.5	up to 30 pts
Answers to self-check questions	up to 10 pts

TIPS FOR THE LAB

- Read error messages from the bottom of the stack trace upwards – the first line that mentions your file is the culprit.
- "Navigator operation requested with a context that does not include a Navigator" usually means you used the `context` of `MaterialApp` itself. Move the button into a separate

widget.

- Never use `BuildContext` after an `await` without checking `if (!context.mounted) return;`.